

1. PROJECT OVERVIEW

- **Project Title:** ComicCraft - AI Comic Story Creator using Gemini Models

2. INTRODUCTION TO EXECUTABLE & RUNTIME ENVIRONMENT :

ComicCraft is built as a lightweight, high-performance web application powered by Python, FastAPI, and Uvicorn. Because it relies on dynamic cloud-based Generative AI APIs (Google Gemini and Hugging Face Stable Diffusion) and local Jinja2 rendering, there is no bulky binary installer required. Instead, the application is executed via a structured Python virtual environment, ensuring all dependencies are isolated and executed seamlessly on any local machine.

3. SYSTEM PREREQUISITES

- **Python Version:** Python 3.10 or higher installed on the host operating system.
- **Operating System:** Windows 10/11 (Compatible with Linux / macOS with minor command adjustments).
- **Internet Connection:** Required for querying external Generative AI endpoints (Gemini & Hugging Face).

4. STEP-BY-STEP EXECUTION GUIDE

Step 1: Open Terminal in Project Directory Navigate to the root folder containing the project files (app, templates, static, requirements.txt, and .env).

Step 2: Create and Activate the Virtual Environment To isolate project dependencies from the global Python installation, run the following commands in the terminal:

```
python -m venv env
```

```
.\env\Scripts\activate # For Windows PowerShell / CMD
```

Step 3: Install Project Dependencies Install all required libraries (FastAPI, Uvicorn, Jinja2, google-generativeai, diffusers, FPDF, Pillow) using the requirements file:

```
pip install -r requirements.txt
```

Step 4: Configure Environment Variables Ensure the .env file is present in the root directory containing your active API credentials:

```
GEMINI_API_KEY=your_google_gemini_api_key_here  
HF_API_KEY=your_huggingface_api_key_here
```

Step 5: Launch the FastAPI Application Server Start the Uvicorn ASGI server to host the application locally with auto-reload enabled:

```
python -m uvicorn app.main:app --reload
```

Step 6: Access the Application Open any modern web browser (Google Chrome, Microsoft Edge, Firefox) and navigate to the local server URL:

<http://127.0.0.1:8000>

5. TROUBLESHOOTING & SETUP FIXES (DEVELOPMENT LOGS) During local execution and environment setup, the system accounts for common OS and network constraints with verified troubleshooting procedures:

- **Issue 1: Windows Long Path Name Error ([WinError 206])**
 - *Cause:* Installing heavy packages like torch or diffusers exceeds Windows' default 260-character path limit.
 - *Solution:* Press Win + R, type regedit, navigate to Computer\HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSe

t\Control\FileSystem, locate LongPathsEnabled, and change its value from 0 to 1.

- **Issue 2: PowerShell Script Execution Policy Restriction (UnauthorizedAccess)**
 - *Cause:* Local PowerShell security settings block virtual environment activation scripts (Activate.ps1).
 - *Solution:* Run Set-ExecutionPolicy RemoteSigned -Scope CurrentUser in the terminal, press Y to confirm permissions, and re-run the activation command.
- **Issue 3: Missing Module Dependencies (ModuleNotFoundError for FastAPI, Uvicorn, Jinja2)**
 - *Cause:* Incomplete package synchronization during initial virtual environment setup.
 - *Solution:* Stop the server using Ctrl + C, reactivate the environment (.env\Scripts\activate), and install missing modules explicitly (e.g., pip install fastapi uvicorn jinja2 or re-run pip install -r requirements.txt).
- **Issue 4: Network DNS Resolution Failure ([Errno 11001] getaddrinfo failed)**
 - *Cause:* Local Internet Service Provider (ISP) network blocks or DNS issues restricting direct connections to external AI endpoints.
 - *Solution:* The application's built-in dual-engine architecture and proxy handlers automatically bypass network blocks, falling back to alternative API routes or generating a local Python PIL comic caption panel to ensure zero downtime.